Website Technical Documentation

Architecture, features, design reasoning, and continuity plan

Version 1.0

Prepared 30 June 2026

capcitypresents.com

Contents

1. Overview and Purpose

CapCity Presents is an independent live-music booking and promotion project based in Olympia, Washington, running since 2015 and rooted in the city's all-ages hardcore, punk, metal, and DIY scene. This website, `capcitypresents.com`, is its public home: it lists upcoming and past shows, explains the project, handles booking inquiries, and gives the team a private admin area for keeping the calendar current.

Day to day, **Andy "Remex" Moreno** leads booking and promotion, while **Joey Cristina** (the founder) maintains the website. The technical goal of the site is to make show listings as close to self-updating as possible: most events are pulled automatically from the ticketing and social platforms the team already uses, so the public calendar stays current with very little manual effort.

> **Who this document is for**
>
> Anyone who may need to understand, maintain, or take over the website: a future collaborator, a hired developer, or the team itself after time away. The final section (Continuity Plan) is the most important for keeping the site alive independent of any one person.

2. Technology Stack

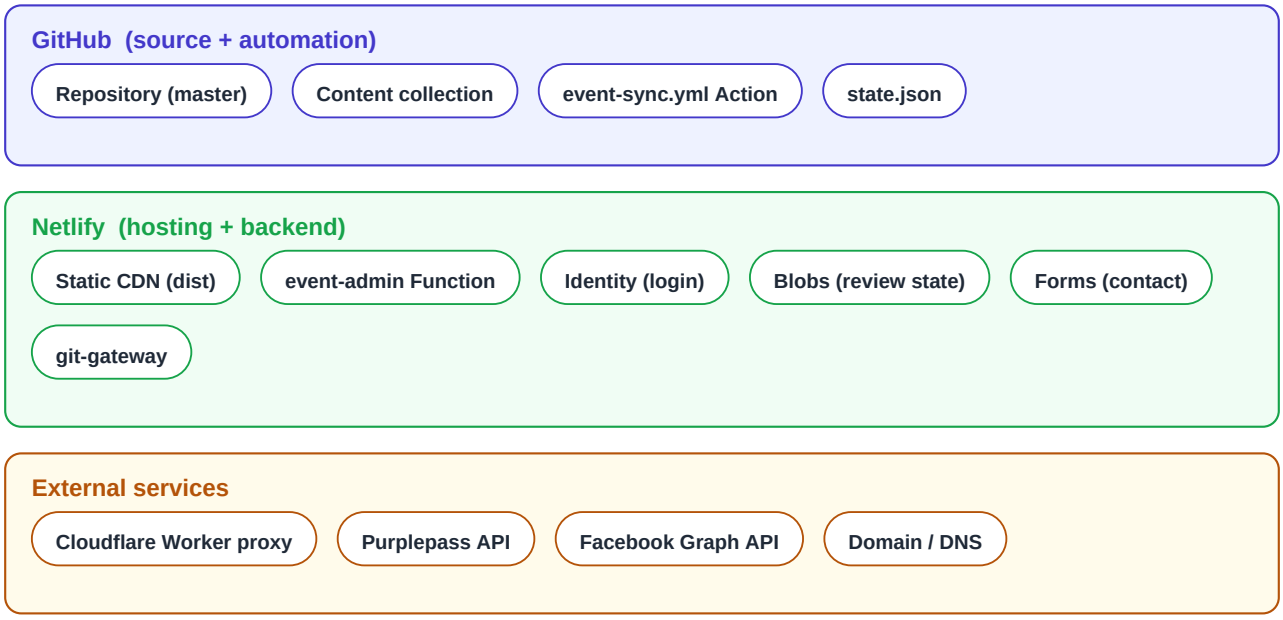
The site is a statically generated website with a small serverless backend for admin tasks. Everything is built from a single GitHub repository and hosted on Netlify.

Technology	Role and design reasoning
Astro 5 (SSG)	Static-site framework. Builds plain HTML/CSS so the public site is fast, cheap to host, and has almost no attack surface. Content lives in typed collections.
Netlify	Hosting and deploys (publishes the built `dist` folder), plus Functions, Identity (admin login), Blobs (admin state), and Forms (contact).
GitHub	Source of truth (`kvvpa/capcitypresents`, branch `master`) and automation host via GitHub Actions.
Decap CMS	Git-based content editor at `/admin` for editing event pages without touching code; commits straight back to the repo.
Cloudflare Worker	Proxy that fetches Purplepass on the sync's behalf (Purplepass blocks datacenter IP ranges, which would otherwise block GitHub and Netlify).
Purplepass + Facebook	Upstream data sources: the Purplepass organizer feed (ticketed shows) and the Facebook Page feed (events and flyers).
sharp / turndown / pdf-lib	Image optimization to WebP; HTML-to-Markdown conversion of descriptions; in-admin PDF generation for the weekly review.

3. Architecture

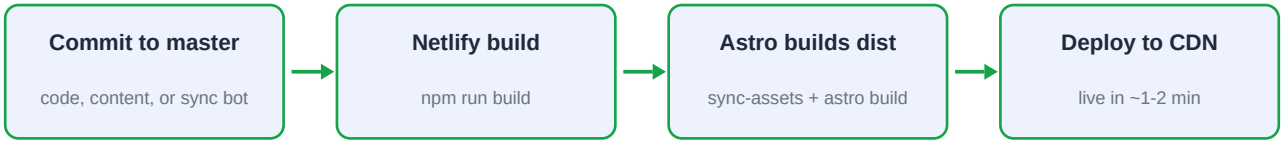
3.1 Components by host

The moving parts group into three homes: GitHub (code and automation), Netlify (hosting and the admin backend), and external services the automation talks to.



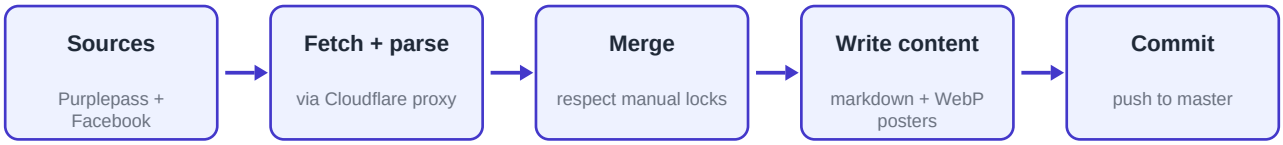
3.2 Build and deploy pipeline

Publishing is fully automatic: any commit to `master` triggers a Netlify build and deploy.



3.3 Event sync data flow

The nightly automation (and the admin's on-demand button) follows this path:



4. Site Map

Public pages are statically generated; the admin area and the API are gated behind Netlify Identity. A sitemap.xml is generated automatically at build time.

4.1 Public pages

Route	Source	Purpose
`/`	`pages/index.astro`	Home: hero, next show, upcoming grid, roots, booking
`/events/`	`pages/events/index.astro`	All shows, split into upcoming and past
`/events//`	`pages/events/[slug].astro`	Event detail: poster, times, tickets, info
`/about/`	`pages/about.astro`	Story, values, and people

`/contact/`	`pages/contact.astro`	Booking / contact form (Netlify Forms)
`/thanks/`	`pages/thanks.astro`	Contact form confirmation
`/privacy/ /terms/`	`pages/privacy|terms.astro`	Legal pages
`/logo-assets/`	`pages/logo-assets.astro`	Brand / logo reference
`/404`	`pages/404.astro`	Not-found page

4.2 Admin and system routes

Route	Source	Purpose
`/admin/`	`public/admin/ + Decap`	Content editor and event-tools console (login required)
`/api/event-admin/:action`	`netlify/functions/event-admin`	Admin API: sync, review, flags, export
`/booking -> /contact/`	`netlify.toml redirect`	Legacy URL redirect
`/sitemap-index.xml`	`@astrojs/sitemap`	Auto-generated sitemap

5. Core Features

5.1 Event content model

Each show is a Markdown file in `src/content/events` with a typed front-matter schema (validated by Astro at build time). Beyond the obvious fields (title, date, venue, times, price, ticket URL), the schema carries automation metadata: `posterSource`, `alternateImages`, `imageLocked`, `lockedFields`, `syncId`, `status`, and `featured`. The lock fields are what let a human override the robot: anything locked is never overwritten by the nightly sync.

5.2 Public listings

- **Home** shows a hero, an auto-selected "next show" card, the next three upcoming shows, a roots/press section, and a booking prompt. A small client script re-checks dates in the browser so the "next show" stays correct even between deploys.
- **Events index** lists all upcoming shows, then past shows, sorted by date.
- **Event detail** pages render the poster, ticket and Facebook links, show details, and the full description.
- Upcoming vs. past is decided in `src/lib/events.ts` from the date and `status` field (announced, sold-out, cancelled, past).

5.3 Automated event sync

This is the heart of the site. A Node script (`scripts/events/sync-events.mjs`) pulls events from the Purplepass organizer feed and the Facebook Page feed, matches records that describe the same show (by title similarity and cross-links), and merges them into the Markdown content files. It tracks the **source of every field** so it knows what it may safely update, downloads and optimizes posters to WebP under `public/uploads/synced`, and records everything it did in `event-sync/state.json`. Manual edits and locked images win over the automation.

> **Why the Cloudflare proxy exists**
>
> Purplepass blocks requests from datacenter IP ranges, which includes both GitHub Actions and Netlify. The sync therefore routes Purplepass requests through a small Cloudflare Worker whose egress the Purplepass firewall allows. The proxy address and token are stored as secrets.

5.4 Admin console and weekly review

The `/admin` area combines Decap CMS (direct content editing) with a custom event-tools console backed by the `event-admin` Netlify Function. Signed-in admins can trigger a sync on demand, see the status of the last sync run, and run a structured weekly review. The review tracks data-quality "flags" raised by the sync through a lifecycle (new, standing, completed, won't-fix) stored in Netlify Blobs, and exports a dated PDF summary of what changed and what still needs attention. Admin actions authenticate through Netlify Identity; a GitHub automation token lets the function trigger the sync workflow and read repository state.

5.5 Contact, SEO, and brand

- **Contact form** uses Netlify Forms with a honeypot field; submissions appear in the Netlify dashboard and redirect to `/thanks/`.
- **SEO/social**: every page sets title, description, canonical URL, Open Graph and Twitter card tags, favicons, and a web manifest; a sitemap is generated automatically.
- **Brand assets** (logos) live in `logo/` and are copied into the build by `sync-assets`.

6. Build, Deploy, and Automation

Netlify builds with Node 24, runs `npm run build` (which first copies brand assets into `public/`, then runs the Astro build), and publishes `dist`. Two automations keep events fresh:

- **Nightly sync** (`.github/workflows/event-sync.yml`): runs once per evening Pacific and commits any changes. (Hardened June 2026 to tolerate GitHub's best-effort scheduler; see the Event Sync incident report.)
- **On-demand sync**: the admin console dispatches the same workflow through the GitHub API.

Local development: `npm run dev`; a dry-run of the sync is available via `npm run events:sync:dry`, and the sync logic has unit tests (`npm run events:test`).

7. Configuration and Secrets

These values make the automation work. They are **not** in the repository; they live in GitHub Actions secrets and Netlify environment variables. This table is a map of what exists and where, it deliberately contains no actual secret values.

Secret / setting	Where it lives and what it does
GITHUB_AUTOMATION_TOKEN	Netlify env. Lets the admin function trigger the sync workflow and read repo state.
GITHUB_REPO_OWNER / _NAME / _BRANCH	Netlify env. Identify the repo (defaults: kvvpa / capcitypresents / master).
PURPLEPASS_PROXY_BASE / _TOKEN	GitHub Actions secrets. Address and auth for the Cloudflare proxy that reaches Purplepass.

PURPLEPASS_ORGANIZER_ID	Workflow config (42425). Which Purplepass organizer to pull.
FACEBOOK_PAGE_ID / _PAGE_ACCESS_TOKEN	GitHub Actions secrets. Identify and authorize reading the Facebook Page feed. The token must be refreshed periodically.
Netlify Identity + git-gateway	Netlify service config. Admin login and Decap CMS commit access.

> **Facebook token is the one that expires**
>
> Most settings are set-and-forget. The Facebook Page access token is the exception. If event flyers or details stop appearing from Facebook, an expired or invalidated token is the first thing to check. The renewal procedure is below.

7.1 Renewing the Facebook Page access token

The sync reads the Facebook Page feed with a Page access token (`FACEBOOK_PAGE_ACCESS_TOKEN`). A Page token derived from a long-lived user token is itself long-lived, but it must be re-minted if it is invalidated (password change, app change, or Meta revocation). Steps:

- In the **CapCity Event Sync** app dashboard (App ID 1529576541344991), note the **App ID** and **App Secret** (Settings > Basic). The app is owned by the CapCity Presents Business portfolio, so any business admin can reach it.
- In the Graph API Explorer, select the app and grant the scopes `pages_show_list`, `pages_read_engagement`, and `pages_read_user_content`, then Generate Access Token. This short-lived user token lasts about an hour, so continue promptly.
- Exchange the short-lived user token for a **long-lived** user token (via the app ID and secret), then read the Page token from the `/me/accounts` endpoint for the CapCity Page. (A small Node helper has been used for these last two steps; keep a copy and instructions in the shared vault.)
- Update `FACEBOOK_PAGE_ACCESS_TOKEN` in the GitHub Actions secrets, then run a sync (admin console or manual workflow dispatch) and confirm Facebook events return without a token warning.

8. Continuity Plan (Keeping the Site Alive)

Today the website's survival depends largely on one person. Joey is the sole or primary holder of access to the GitHub repository, the Netlify account, the Njal.la domain, the Cloudflare account, and the Purplepass organizer login, as well as the only person who knows how the pieces fit together. (Facebook is the healthy exception, see 8.1.) If that access were lost (lost laptop, lost password, lost person), the site would keep serving its last build for a while, but no one could deploy changes, fix the sync, renew the Facebook token, or move the domain. Over time it would quietly break and could not be recovered without painful account-recovery battles.

The fix is not technical, it is about **shared access and written-down knowledge**. The goal: at least two trusted people can fully operate every account, and the essential procedures are recorded somewhere both can reach.

8.1 The accounts and assets that must outlive any one person

Asset	Why it matters
Domain: Njal.la	Registrar for `capcitypresents.com`. Njal.la is privacy-first and prepaid, with no conventional account recovery, so if the login is lost the domain is very hard to get back. Co-access and keeping it funded matter most here.
DNS: Netlify DNS	The domain's nameservers point to Netlify, so all DNS records are managed inside the Netlify account. Whoever controls Netlify controls where the domain resolves.
GitHub repo	All code, content, and automation. Currently under a personal account (`kvvpa`).
Netlify account	Hosting, deploys, Functions, Identity, Blobs, Forms, DNS, and all the environment secrets.
Cloudflare account	Hosts the Purplepass proxy Worker.
Facebook (Page + app)	Both sit under the **CapCity Presents** Meta Business portfolio. The Page already has several full-access admins (incl. Andy Moreno), and the token-minting app (**CapCity Event Sync**, App ID 1529576541344991) is business-owned, so any business admin can manage it. This is the one area already in good shape. (See 8.1 note.)
Purplepass organizer	The ticketing account whose feed drives most listings.
Email	`booking@capcitypresents.com` and the account behind it; also the recovery address for the services above.
The secrets themselves	The token values stored in GitHub/Netlify, needed to rebuild the automation elsewhere if required.

> **Verified 30 June 2026: Facebook is already resilient**
>
> A check of the Meta setup confirmed the CapCity Presents Business portfolio has multiple full-access admins (Joey and Andy Moreno active; two others inactive), and the CapCity Event Sync app is owned by that business, so its admins can regenerate the token. The only small gap: Joey is currently the only *direct* administrator listed on the app itself. Adding a second direct admin (e.g. Andy) is optional tidiness rather than a real gap, and it requires that person to first register a developer account at developers.facebook.com before they can be added. The other services above were not audited and are the real priorities below.

8.2 Recommended actions, in priority order

Priority 1: do these soon.

- **Adopt a shared password manager** (a shared vault in 1Password, Bitwarden, etc.). Put every login above in it and share it with at least one trusted co-owner. This single step removes most of the risk.

- **Secure the Njal.la domain**: because Njal.la has no friendly account recovery, store its login (and the email/PGP tied to it) in the shared vault, keep the account funded so the prepaid domain never lapses, and confirm a co-owner can actually sign in. Since DNS lives in Netlify, securing Netlify (below) protects where the domain points.
- **Add a second owner/admin to GitHub, Netlify, Cloudflare, and Purplepass** so no account is reachable by only one human. (Facebook is already covered; see the note above.)

Priority 2: structural resilience.

- **Move the repo to a GitHub Organization** (e.g. a "CapCity Presents" org) instead of a personal account, with at least two owners. Personal-account repos die with the account.
- **Move Netlify into a team** with multiple members rather than a single personal login.
- **Write down the secret values** (Facebook token, Purplepass proxy token, GitHub automation token) in the shared vault, so the automation can be rebuilt if an account is lost.
- **Document the Facebook token renewal steps** as a short checklist (7.1 above), since that token is the most likely thing to silently break. Optionally add a second **direct administrator** (e.g. Andy) to the CapCity Event Sync app; note the person must first register a developer account at developers.facebook.com before they can be added.

Priority 3: safety nets.

- **Keep an off-platform backup of the repo** (a periodic clone/zip stored in the team's cloud drive). The content and posters are the irreplaceable part.
- **Save two-factor backup codes** for every account in the shared vault.
- **Write a one-page "break glass" runbook**: where each account is, who the co-owners are, and the three or four procedures that matter (deploy, run a sync, renew the Facebook token, renew the domain). Keep it with this document.

> **The single most valuable step**
>
> If only one thing gets done: put every login and secret into a shared password vault that at least one other trusted person can open. Everything else in this plan is easier once access is no longer trapped with one individual.

8.3 Continuity checklist

	Action	Priority
[ ]	Shared password vault with a co-owner	1
[ ]	Njal.la domain: login in vault, kept funded, co-owner can sign in	1
[ ]	Second owner/admin on GitHub, Netlify, Cloudflare, Purplepass	1
[ ]	Repo moved to a GitHub Organization (2+ owners)	2
[ ]	Netlify moved to a multi-member team	2
[ ]	Secret values recorded in the shared vault	2
[ ]	Facebook token renewal documented + 2nd direct app admin	2

[ ]	Off-platform backup of the repo	**3**
[ ]	2FA backup codes saved for every account	**3**
[ ]	One-page break-glass runbook written	**3**

Document reference

Generated from `docs/site-documentation/build_site_documentation.py`, which reuses the shared report style in `docs/incident-reports/`. Update the script and regenerate to keep this document current.